

HANDS-ON WORKSHEET

AI-Assisted Debugging: Stack Trace vs. AI

Curiosity Trigger: “Can AI explain a bug better than a stack trace can?”

Learning goal: Students will distinguish between runtime evidence and AI-generated explanations, identify logical bugs, write strong debugging prompts, and verify AI suggestions through testing.

Name	Reg. No.	Batch	Date
Chandan Kumar	2503A51664	66	03/09/2026

Activity 1 – PREDICT

Before running the code or asking AI, make a prediction. The program below contains a logical bug. It runs without producing a Python error, but the result can be wrong.

```
def calculate_discount(price, is_member):
    discount = 0

    if is_member:
        discount = 10
    else:
        discount = 0

    final_price = price discount / 100
    return final_price
```

```
print(calculate_discount(1000, True))
```

Predict before testing:

- What do you think the output should be for price = 1000 and is_member = True?

Answer: 900

- Do you expect a stack trace? Why or why not?

Answer: No. The worksheet states that the program runs without producing a Python error; the problem is a logical error in the calculation.

- What do you think is causing the problem?

Answer: The final_price calculation is wrong. The 10% discount should be subtracted from the original price, not used as the final price calculation.

Activity 2 – EXPERIMENT: Run and Observe

Run the program in VS Code/Python. Record the actual output. Notice whether a stack trace appears.

Test	Student Record
------	----------------

Input	price = 1000, is_member = True
Actual output	100
Stack trace?	No — no stack trace was produced.

Activity 3 – Ask AI a WEAK Prompt

Use this prompt exactly:

“My code is not working, fix it.”

Record what AI suggests:

AI diagnosis: The discount calculation is using the discount percentage as the final price calculation instead of subtracting the discount amount from the original price.

AI fix: Calculate the discount amount first and subtract it from price: `final_price = price - (price * discount / 100)`.

Do you trust the answer immediately? Yes / No

Why? Because an AI-generated answer should be checked against the code and verified by testing rather than trusted immediately.

Activity 4 – Ask AI a STRONG Prompt

Now provide context, code, input, actual output, expected output, and a specific instruction.

Here is my `calculate_discount()` code.

Input: price = 1000, is_member = True

Actual output: 100

Expected: A 10% member discount should be applied, so the result should be 900.

Error/stack trace: No error message was produced.

Find the logical bug.

Explain the ROOT CAUSE first.

Then provide the corrected code.

Finally, explain how I can test that the fix is correct.

Record AI's response:

Root cause: The calculation does not subtract the discount amount from the original price. A 10% discount on 1000 should reduce the price by 100.

Suggested fix: `final_price = price - (price * discount / 100)`

Suggested test: Test (1000, True) → 900, (1000, False) → 1000, and (500, True) → 450.

Activity 5 – Verify AI Instead of Trusting AI

Modify the code only after understanding the explanation. Then test both member and non-member cases.

Test Case	Expected	Actual	Pass/Fail
1000, True	900	900	Pass
1000, False	1000	1000	Pass
500, True	450	450	Pass

Activity 6 – Stack Trace Challenge

Run this code and study the stack trace:

```
def book_slot(doc_id):
    doctors = {
        "CARDIO-01": "Dr. A",
        "CARDIO-02": "Dr. B"
    }
    return "Booking confirmed for " + doctors[doc_id]

print(book_slot("CARDIO-07"))
```

Answer:

1. What is the exception type?

KeyError

2. Which line caused the error?

The error occurs on the return line: `return "Booking confirmed for " + doctors[doc_id]`

3. What does the stack trace tell you?

It tells you that the program raised a `KeyError` and identifies where the invalid dictionary lookup occurred.

4. What does it NOT tell you about the intended business logic?

It does not tell you the intended business rule for an unknown doctor ID or what the application should do in that situation.

5. Write a strong AI prompt asking for the root cause and a safe fix.

This Python program books a doctor using a doctor ID. The input is "CARDIO-07", which is not present in the doctors dictionary. Explain the ROOT CAUSE first, identify the evidence in the code, propose a safe fix for an unknown ID, and give tests for both valid and invalid doctor IDs.

Activity 7 – AI Can Be Wrong

Ask AI to diagnose the previous problem. Then deliberately challenge the response.

Use this verification prompt:

“Do not assume your first diagnosis is correct. Identify the evidence in the code that supports your diagnosis. Give me one test that could prove your diagnosis wrong.”

AI pointed out that `doctors[doc_id]` performs a dictionary lookup and "CARDIO-07" is not one of the available keys.

Test a valid ID such as "CARDIO-01" and an invalid ID such as "CARDIO-07" to see whether the diagnosis matches the program behavior.

Did your test confirm the diagnosis? Yes / No

I learned that an AI explanation must be checked against the actual code and verified with tests; a confident answer is not automatically correct.

Activity 8 – Create Your Own Debugging Prompt

Create a strong prompt for a logical bug in a small Python program. Your prompt must contain all six elements:

- Context / purpose of the program
- Relevant code
- Input
- Actual output or behavior
- Expected output or behavior
- Instruction: explain ROOT CAUSE first and propose a test

Your prompt:

Context: This program calculates a student's discount based on membership.

Code: `final_price = price * discount / 100`

Input: price = 1000, discount = 10. Actual output = 100. Expected output = 900.

Explain the ROOT CAUSE first, then give the corrected code and propose one test that could prove your diagnosis wrong.

Activity 9 – Predict → Experiment → Record Reflection

Stage	Your observation
Predict	I expected the member price to be 900 and expected no stack trace because the worksheet describes a logical bug.
Experiment	The original calculation produced 100. After correcting the formula, the member and non-member tests produced the expected results.
Record	AI was correct when it identified the incorrect discount calculation. The evidence was the formula itself, and testing the corrected formula confirmed it.

Key Takeaway

STACK TRACE = evidence about where/how execution failed

AI = a reasoning assistant that may explain possible causes

TESTING = the final check that determines whether the explanation and fix are correct

Student challenge: Never accept an AI-generated fix only because it sounds confident. Understand the root cause, test the proposed fix, and record the evidence.